HOMEWORK – 6

1. Simulate the undo and redo operation in work packages using a doubly linked list
   CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
   char work[50];
   struct Node *prev, *next;
};

struct Node *current = NULL;

void addWork(char work[]) {
   struct Node *newNode = malloc(sizeof(struct Node));

   strcpy(newNode->work, work);
   newNode->prev = current;
   newNode->next = NULL;

   if (current != NULL) {
      current->next = newNode;
   }

   current = newNode;
   printf("Work added: %s\n", work);
}

void undo() {
   if (current == NULL) {
      printf("Nothing to undo\n");
```

```
        return;
    }

    printf("Undo: %s\n", current->work);
    current = current->prev;
}

void redo() {
    if (current == NULL) {
        printf("Nothing to redo\n");
        return;
    }

    if (current->next == NULL) {
        printf("Nothing to redo\n");
        return;
    }

    current = current->next;
    printf("Redo: %s\n", current->work);
}

void display() {
    struct Node *temp = current;

    if (temp == NULL) {
        printf("No active work\n");
        return;
    }

    printf("Current work: %s\n", temp->work);
}

int main() {
    addWork("Package A");
```

```
    addWork("Package B");
    addWork("Package C");

    display();

    undo();
    display();

    undo();
    display();

    redo();
    display();

    return 0;
}
```

OUTPUT:
Work added: Package A
Work added: Package B
Work added: Package C
Current work: Package C
Undo: Package C
Current work: Package B
Undo: Package B
Current work: Package A
Redo: Package B
Current work: Package B

2. Given two sorted losts L1 and L2. Write a procedure to compute L1 intersection L2 and L1 union L2 using basic list operations.
   CODE:

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct node {
   int data;
   struct node *next;
};

struct node *create(int x) {
   struct node *n = malloc(sizeof(struct node));
   n->data = x;
   n->next = NULL;
   return n;
}

void display(struct node *p) {
   while (p != NULL) {
      printf("%d ", p->data);
      p = p->next;
   }
   printf("\n");
}

int main() {
   struct node *L1 = create(1);
   L1->next = create(3);
   L1->next->next = create(5);
   L1->next->next->next = create(7);

   struct node *L2 = create(3);
   L2->next = create(5);
   L2->next->next = create(6);
   L2->next->next->next = create(8);

   struct node *p = L1, *q = L2;

   printf("Intersection: ");
```

```
while (p != NULL && q != NULL) {
   if (p->data == q->data) {
      printf("%d ", p->data);
      p = p->next;
      q = q->next;
   }
   else if (p->data < q->data)
      p = p->next;
   else
      q = q->next;
}

p = L1;
q = L2;

printf("\nUnion: ");
while (p != NULL && q != NULL) {
   if (p->data < q->data) {
      printf("%d ", p->data);
      p = p->next;
   }
   else if (p->data > q->data) {
      printf("%d ", q->data);
      q = q->next;
   }
   else {
      printf("%d ", p->data);
      p = p->next;
      q = q->next;
   }
}

while (p != NULL) {
   printf("%d ", p->data);
   p = p->next;
```

```
    }

    while (q != NULL) {
        printf("%d ", q->data);
        q = q->next;
    }

    return 0;
}
```

OUTPUT:
Intersection: 3 5
Union: 1 3 5 6 7 8

3. Swap two elements by adjusting only the pointers (and not the data) using doubly linked list
   CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *prev, *next;
};

struct node *create(int x) {
    struct node *n = malloc(sizeof(struct node));
    n->data = x;
    n->prev = n->next = NULL;
    return n;
}

void display(struct node *head) {
    while (head != NULL) {
```

```
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main() {
    struct node *a = create(10);
    struct node *b = create(20);
    struct node *c = create(30);

    a->next = b;
    b->prev = a;
    b->next = c;
    c->prev = b;

    struct node *head = a;

    printf("Before swapping: ");
    display(head);

    // Swap adjacent nodes a and b by adjusting pointers
    a->next = c;
    c->prev = a;

    b->prev = NULL;
    b->next = a;
    a->prev = b;

    head = b;

    printf("After swapping: ");
    display(head);

    return 0;
```

```
}
```

OUTPUT:

Before swapping: 10 20 30

After swapping: 20 10 30

4.Write a C program to reverse a given doubly linked list

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *prev, *next;
};

struct node *create(int x) {
   struct node *n = malloc(sizeof(struct node));
   n->data = x;
   n->prev = NULL;
   n->next = NULL;
   return n;
}

void display(struct node *head) {
   while (head != NULL) {
      printf("%d ", head->data);
      head = head->next;
   }
   printf("\n");
}

struct node *reverse(struct node *head) {
```

```
    struct node *temp, *current = head;

    while (current != NULL) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;

        head = current;
        current = current->prev;
    }

    return head;
}

int main() {
    struct node *head = create(10);
    head->next = create(20);
    head->next->prev = head;
    head->next->next = create(30);
    head->next->next->prev = head->next;

    printf("Original list: ");
    display(head);

    head = reverse(head);

    printf("Reversed list: ");
    display(head);

    return 0;
}
```

OUTPUT:

```
Original list: 10 20 30
Reversed list: 30 20 10
```

5.Write a C program to delete the middle element of a double linked list

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *prev, *next;
};

struct node *create(int x) {
   struct node *n = malloc(sizeof(struct node));
   n->data = x;
   n->prev = n->next = NULL;
   return n;
}

struct node *deleteMiddle(struct node *head) {
   struct node *slow = head, *fast = head;

   if (head == NULL)
      return NULL;

   while (fast != NULL && fast->next != NULL) {
      slow = slow->next;
      fast = fast->next->next;
   }

   if (slow->prev != NULL)
      slow->prev->next = slow->next;
   else
      head = slow->next;
```

```
    if (slow->next != NULL)
        slow->next->prev = slow->prev;

    free(slow);
    return head;
}

void display(struct node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main() {
    struct node *head = NULL, *temp, *newNode;
    int n, x, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &x);

        newNode = create(x);

        if (head == NULL)
            head = newNode;
        else {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;
```

```
            temp->next = newNode;
            newNode->prev = temp;
        }
    }

    printf("Original list: ");
    display(head);

    head = deleteMiddle(head);

    printf("After deleting middle: ");
    display(head);

    return 0;
}
```

OUTPUT:

```
Enter number of nodes: 5
Enter data: 10
Enter data: 20
Enter data: 30
Enter data: 40
Enter data: 50
Original list: 10 20 30 40 50
After deleting middle: 10 20 40 50
```

6. Remove duplicate elements from a 1.sorted DLL 2. unsorted DLL

CODE:

SORTED:

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct node {
   int data;
   struct node *prev, *next;
};

int main() {
   struct node *head = NULL, *temp, *n, *p;
   int i, count, x;

   printf("Enter number of nodes: ");
   scanf("%d", &count);

   for (i = 0; i < count; i++) {
      scanf("%d", &x);
      n = malloc(sizeof(struct node));
      n->data = x;
      n->prev = n->next = NULL;

      if (head == NULL)
         head = n;
      else {
         temp = head;
         while (temp->next != NULL)
            temp = temp->next;
         temp->next = n;
         n->prev = temp;
      }
```

```
    }

    temp = head;
    while (temp != NULL && temp->next != NULL) {
        if (temp->data == temp->next->data) {
            p = temp->next;
            temp->next = p->next;

            if (p->next != NULL)
                p->next->prev = temp;

            free(p);
        } else
            temp = temp->next;
    }

    printf("After removing duplicates: ");
    for (temp = head; temp != NULL; temp = temp->next)
        printf("%d ", temp->data);

    return 0;
}
```

Output:

6

1 1 2 3 3 4

After removing duplicates: 1 2 3 4

Unsorted:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *prev, *next;
};

int main() {
   struct node *head = NULL, *temp, *n, *p, *q;
   int i, count, x;

   printf("Enter number of nodes: ");
   scanf("%d", &count);

   for (i = 0; i < count; i++) {
      scanf("%d", &x);
      n = malloc(sizeof(struct node));
      n->data = x;
      n->prev = n->next = NULL;

      if (head == NULL)
         head = n;
      else {
         temp = head;
         while (temp->next != NULL)
            temp = temp->next;
```

```
      temp->next = n;
      n->prev = temp;
    }
  }

  temp = head;
  while (temp != NULL) {
    q = temp->next;

    while (q != NULL) {
      if (temp->data == q->data) {
        p = q;
        q = q->next;

        if (p->prev != NULL)
          p->prev->next = p->next;

        if (p->next != NULL)
          p->next->prev = p->prev;

        free(p);
      } else
        q = q->next;
    }

    temp = temp->next;
  }
```

```
    printf("After removing duplicates: ");
    for (temp = head; temp != NULL; temp = temp->next)
        printf("%d ", temp->data);

    return 0;
}
```

Output:

6

4 2 4 1 2 3

After removing duplicates: 4 2 1 3

7. calculate the frequency count of a given integer in a circular DLL

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *prev, *next;
};

int main() {
    struct node *head = NULL, *temp, *newNode;
    int n, x, key, count = 0, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
```

```
    printf("Enter data: ");
    scanf("%d", &x);

    newNode = malloc(sizeof(struct node));
    newNode->data = x;

    if (head == NULL) {
      head = newNode;
      head->next = head;
      head->prev = head;
    }
    else {
      temp = head->prev;

      newNode->next = head;
      newNode->prev = temp;

      temp->next = newNode;
      head->prev = newNode;
    }
}

printf("Enter integer to find frequency: ");
scanf("%d", &key);

if (head != NULL) {
    temp = head;
```

```
    do {
      if (temp->data == key)
        count++;

      temp = temp->next;

    } while (temp != head);
  }

  printf("Frequency of %d = %d", key, count);

  return 0;
}
```

Output:

```
Enter number of nodes: 5
Enter data: 10
Enter data: 20
Enter data: 10
Enter data: 30
Enter data: 10
Enter integer to find frequency: 10
Frequency of 10 = 3
```

8.Search for a particular element in a binary tree

CODE:

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct node {
   int data;
   struct node *left, *right;
};

struct node *create(int x) {
   struct node *n = malloc(sizeof(struct node));
   n->data = x;
   n->left = n->right = NULL;
   return n;
}

int search(struct node *root, int key) {
   if (root == NULL)
      return 0;

   if (root->data == key)
      return 1;

   return search(root->left, key) ||
        search(root->right, key);
}

int main() {
   struct node *root = create(10);
   root->left = create(20);
   root->right = create(30);
   root->left->left = create(40);
```

```
  root->left->right = create(50);

  int key;

  printf("Enter element to search: ");
  scanf("%d", &key);

  if (search(root, key))
    printf("Element found");
  else
    printf("Element not found");

  return 0;
}
```

OUTPUT:

Enter element to search: 40

Element found

9. Calculate the height or depth of a tree and print it

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
  int data;
  struct node *left, *right;
};
```

```
struct node *create(int x) {
   struct node *n = malloc(sizeof(struct node));
   n->data = x;
   n->left = n->right = NULL;
   return n;
}

int height(struct node *root) {
   int lh, rh;

   if (root == NULL)
      return 0;

   lh = height(root->left);
   rh = height(root->right);

   if (lh > rh)
      return lh + 1;
   else
      return rh + 1;
}

int main() {
   struct node *root = create(10);

   root->left = create(20);
   root->right = create(30);
```

```
    root->left->left = create(40);
    root->left->right = create(50);

    printf("Height of tree = %d", height(root));

    return 0;
}
```

OUTPUT:

Height of tree = 3

10. Given a binary tree, convert this to a doubly linked list keeping the same order (inorder traversal)

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left, *right;
};

struct node *prev = NULL;

struct node *create(int x) {
    struct node *n = malloc(sizeof(struct node));
    n->data = x;
    n->left = n->right = NULL;
```

```
    return n;
}

void convert(struct node *root) {
    if (root == NULL)
        return;

    convert(root->left);

    root->left = prev;

    if (prev != NULL)
        prev->right = root;

    prev = root;

    convert(root->right);
}

int main() {
    struct node *root = create(10);

    root->left = create(20);
    root->right = create(30);
    root->left->left = create(40);
    root->left->right = create(50);

    convert(root);
```

```
  struct node *head = prev;

  while (head->left != NULL)
    head = head->left;

  printf("Doubly Linked List: ");

  while (head != NULL) {
    printf("%d ", head->data);
    head = head->right;
  }

  return 0;
}
```

OUTPUT:

Doubly Linked List: 40 20 50 10 30

11. given a binary tree the task is to connect the nodes that are at the same level using next right pointer

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
  int data;
  struct node *left, *right, *nextRight;
```

```
};

struct node *create(int x) {
   struct node *n = malloc(sizeof(struct node));

   n->data = x;
   n->left = n->right = n->nextRight = NULL;

   return n;
}

void connect(struct node *root) {
   struct node *temp, *nextLevel;

   if (root == NULL)
      return;

   root->nextRight = NULL;
   temp = root;

   while (temp != NULL) {
      nextLevel = NULL;
      struct node *prev = NULL;

      while (temp != NULL) {
         if (temp->left != NULL) {
            if (prev == NULL)
               nextLevel = temp->left;
```

```
            else
               prev->nextRight = temp->left;

            prev = temp->left;
         }

         if (temp->right != NULL) {
            if (prev == NULL)
               nextLevel = temp->right;
            else
               prev->nextRight = temp->right;

            prev = temp->right;
         }

         temp = temp->nextRight;
      }

      if (prev != NULL)
         prev->nextRight = NULL;

      temp = nextLevel;
   }
}

void display(struct node *root) {
   struct node *level = root;
```

```
    while (level != NULL) {
        struct node *temp = level;

        while (temp != NULL) {
            printf("%d -> ", temp->data);
            temp = temp->nextRight;
        }

        printf("NULL\n");

        if (level->left != NULL)
            level = level->left;
        else if (level->right != NULL)
            level = level->right;
        else
            break;
    }
}

int main() {
    struct node *root = create(1);

    root->left = create(2);
    root->right = create(3);

    root->left->left = create(4);
    root->left->right = create(5);
```

```
    root->right->left = create(6);

    connect(root);

    printf("Nodes connected at same level:\n");
    display(root);

    return 0;
}
```

OUTPUT:

Nodes connected at same level:

1 -> NULL

2 -> 3 -> NULL

4 -> 5 -> 6 -> NULL

12. Implement priority queue using binary search tree and perform operation insert

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left, *right;
};

struct node *create(int x) {
```

```
    struct node *n = malloc(sizeof(struct node));

    n->data = x;
    n->left = n->right = NULL;

    return n;
}

struct node *insert(struct node *root, int x) {
    if (root == NULL)
        return create(x);

    if (x < root->data)
        root->left = insert(root->left, x);
    else
        root->right = insert(root->right, x);

    return root;
}

void inorder(struct node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
```

```
int main() {
   struct node *root = NULL;
   int n, x, i;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++) {
      printf("Enter element: ");
      scanf("%d", &x);

      root = insert(root, x);
   }

   printf("Priority Queue (Inorder): ");
   inorder(root);

   return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter element: 40
Enter element: 20
Enter element: 60
Enter element: 10
Enter element: 30
Priority Queue (Inorder): 10 20 30 40 60
```

13. In a given bst write routines to delete a node with no child , two child, and oine child.

CODE:

```
#include <stdio.h>

#include <stdlib.h>


struct node {

  int data;

  struct node *left, *right;

};


struct node *create(int x) {

  struct node *n = malloc(sizeof(struct node));


  n->data = x;

  n->left = n->right = NULL;


  return n;

}


struct node *insert(struct node *root, int x) {

  if (root == NULL)

    return create(x);


  if (x < root->data)

    root->left = insert(root->left, x);

  else
```

```
        root->right = insert(root->right, x);

    return root;
}

struct node *findMin(struct node *root) {
    while (root->left != NULL)
        root = root->left;

    return root;
}

struct node *deleteNode(struct node *root, int key) {
    struct node *temp;

    if (root == NULL)
        return NULL;

    if (key < root->data)
        root->left = deleteNode(root->left, key);

    else if (key > root->data)
        root->right = deleteNode(root->right, key);

    else {
        // Case 1: No child
        if (root->left == NULL && root->right == NULL) {
            free(root);
```

```
        return NULL;
      }

      // Case 2: One child
      else if (root->left == NULL) {
        temp = root->right;
        free(root);
        return temp;
      }

      else if (root->right == NULL) {
        temp = root->left;
        free(root);
        return temp;
      }

      // Case 3: Two children
      temp = findMin(root->right);

      root->data = temp->data;

      root->right = deleteNode(root->right, temp->data);
    }

    return root;
}

void inorder(struct node *root) {
```

```
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main() {
    struct node *root = NULL;
    int n, x, key, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter element: ");
        scanf("%d", &x);

        root = insert(root, x);
    }

    printf("Inorder before deletion: ");
    inorder(root);

    printf("\nEnter element to delete: ");
    scanf("%d", &key);

    root = deleteNode(root, key);
```

```
    printf("Inorder after deletion: ");
    inorder(root);

    return 0;
}
```

OUTPUT:

Enter number of nodes: 6

Enter element: 50

Enter element: 30

Enter element: 70

Enter element: 20

Enter element: 40

Enter element: 60

Enter element: 80

Enter element to delete: 50

Inorder before deletion: 20 30 40 50 60 70 80

Inorder after deletion: 20 30 40 60 70 80

14. convert the given postfix expression to an expression tree

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    char data;
```

```
    struct node *left, *right;
};

struct node *stack[100];
int top = -1;

struct node *create(char x) {
    struct node *n = malloc(sizeof(struct node));

    n->data = x;
    n->left = n->right = NULL;

    return n;
}

void push(struct node *n) {
    stack[++top] = n;
}

struct node *pop() {
    return stack[top--];
}

int isOperand(char ch) {
    if ((ch >= 'a' && ch <= 'z') ||
        (ch >= 'A' && ch <= 'Z') ||
        (ch >= '0' && ch <= '9'))
        return 1;
```

```
    return 0;
}

struct node *createTree(char postfix[]) {
    struct node *n, *a, *b;
    int i;

    for (i = 0; postfix[i] != '\0'; i++) {
        if (isOperand(postfix[i])) {
            n = create(postfix[i]);
            push(n);
        }
        else {
            n = create(postfix[i]);

            a = pop();
            b = pop();

            n->right = a;
            n->left = b;

            push(n);
        }
    }

    return pop();
}
```

```
void inorder(struct node *root) {
   if (root != NULL) {
      inorder(root->left);
      printf("%c", root->data);
      inorder(root->right);
   }
}

int main() {
   char postfix[100];
   struct node *root;

   printf("Enter postfix expression: ");
   scanf("%s", postfix);

   root = createTree(postfix);

   printf("Infix expression: ");
   inorder(root);

   return 0;
}
```

OUTPUT:

Enter postfix expression: ab+c*

Infix expression: a+b*c

15. Construct bst using 1.inorder 2.preorder 3.postorder node representation

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node *left;
   struct node *right;
};

struct node* createNode(int value)
{
   struct node *newnode;

   newnode = (struct node*)malloc(sizeof(struct node));

   newnode->data = value;
   newnode->left = NULL;
   newnode->right = NULL;

   return newnode;
}

struct node* insert(struct node *root, int value)
{
   if (root == NULL)
      return createNode(value);
```

```
    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);

    return root;
}

void inorder(struct node *root)
{
    if (root != NULL)
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main()
{
    struct node *root = NULL;

    int preorder[] = {50, 30, 20, 40, 70, 60, 80};
    int n = 7;
    int i;

    for (i = 0; i < n; i++)
```

```
    root = insert(root, preorder[i]);

  printf("Inorder traversal: ");
  inorder(root);

  return 0;
}
```

OUTPUT:

Inorder traversal: 20 30 40 50 60 70 80

16.Implement heap sort for a set of numbers.

CODE:

```
#include <stdio.h>

#define MAX 100

struct Heap
{
  int a[MAX];
  int size;
};

void heapify(struct Heap *h, int n, int i)
{
  int largest = i;
  int left = 2 * i + 1;
  int right = 2 * i + 2;
```

```
    int temp;

    if (left < n && h->a[left] > h->a[largest])
        largest = left;

    if (right < n && h->a[right] > h->a[largest])
        largest = right;

    if (largest != i)
    {
        temp = h->a[i];
        h->a[i] = h->a[largest];
        h->a[largest] = temp;

        heapify(h, n, largest);
    }
}

void heapSort(struct Heap *h)
{
    int i, temp;

    // Build max heap
    for (i = h->size / 2 - 1; i >= 0; i--)
        heapify(h, h->size, i);

    // Extract elements from heap
    for (i = h->size - 1; i > 0; i--)
```

```
    {
        temp = h->a[0];
        h->a[0] = h->a[i];
        h->a[i] = temp;

        heapify(h, i, 0);
    }
}

void display(struct Heap h)
{
    int i;

    for (i = 0; i < h.size; i++)
        printf("%d ", h.a[i]);

    printf("\n");
}

int main()
{
    struct Heap h;
    int i;

    printf("Enter number of elements: ");
    scanf("%d", &h.size);

    printf("Enter the elements:\n");
```

```
    for (i = 0; i < h.size; i++)
        scanf("%d", &h.a[i]);

    heapSort(&h);

    printf("Sorted elements:\n");
    display(h);

    return 0;
}
```

OUTPUT:

Enter number of elements: 5

Enter the elements:

25 10 30 5 15

Sorted elements:

5 10 15 25 30